VitalFlow V1 — AI Scribe Review UI

How the physician reviews, edits, accepts, and rejects an AI-generated SOAP draft + code suggestions from inside the encounter workspace. This doc is the UI counterpart to docs/ai-scribe.md — it describes **only the clinician-facing surface** and how it plugs into the existing workspace.

See also:

- docs/ai-scribe.md — the end-to-end scribe architecture and data model
- docs/encounter-workspace.md — the vertical-card stack the review card lives inside
- docs/permissions-matrix.md — `ai:invoke`, `clinical:write`, `clinical:sign`, `notes:amend`

---

1. UI flow

The AI review surface is **one Card** in the encounter workspace, positioned between the Clinical-note card and the Documents card. It has four visual states, selected server-side from the most recent `ai_scribe_sessions` row for this encounter.

```
State A — Intake
┌─ AI scribe ──────────────────────────────────────┐
│ No draft for this visit.                          │
│ [Upload audio] [Paste transcript]                 │
└──────────────────────────────────────────────────┘
                      │ user starts session
                      ▼
State B — In progress
┌─ AI scribe · generating draft ────────────────────┐
│ ● Transcribe      ✓ 18s                           │
│ ● Synthesize      … running                       │
│ ○ Suggest codes                                   │
│ [Refresh status] [Cancel]                         │
└──────────────────────────────────────────────────┘
                      │ session.status → awaiting_review
                      ▼
State C — Review (the main surface, yellow-tinted "AI DRAFT")
┌─ AI scribe · DRAFT — not yet in chart ────────────┐
│ ⚠ 2 warnings for physician review                 │
│ ▸ Transcript (N segments)                         │
│ ── Subjective  combined 0.84 · sources (2) ──     │
│   [editable textarea]                             │
│   sources: seg-11a4, seg-22b8                     │
│ ── Objective  combined 0.66 ⚠ Review ──           │
│ ...                                               │
│ ── Code suggestions ──                            │
│ ☐ J02.9  Acute pharyngitis, unspecified  0.81     │
│     why: ...                                      │
│     missing: streptococcal test result            │
│ [Accept into note as draft] [Reject...]           │
└──────────────────────────────────────────────────┘
                      │ user clicks Accept
                      ▼
State D — Terminal summary
┌─ AI scribe · accepted into note v3 ──────────────┐
│ Accepted by Dr. Lin · 2026-04-20 14:22            │
│ 2 of 3 suggested codes accepted                   │
└──────────────────────────────────────────────────┘
```

AI-draft vs provider-approved — how we make it visually unmistakable

- **While in state C**, the card has a yellow-tinted background and a persistent `AI DRAFT` chip in the header. Nothing in the card lands in the chart until the physician clicks Accept.
- **On Accept**, the content is INSERTed into `encounter_notes` with `ai_assisted=true` and `ai_request_id` set. The existing Clinical-note card renders a banner ("AI-assisted draft — review and sign") and the version-history list shows an `AI` pill on that version.
- **On Sign**, the note's status flips to `signed` via the existing sign flow. The `ai_assisted` flag stays visible in the audit trail; the user-facing chrome returns to normal provider-approved styling.

2. Component structure

All Server Components. No `"use client"`. State is URL-driven and POST-redirect-GET, matching the existing `saveNoteDraft` / `signNote` / `amendNote` pattern in the workspace.

```
apps/web/src/app/(app)/encounters/[id]/ai-review/
├── AIReviewCard.tsx            top-level state selector (A|B|C|D)
├── AIReviewIntakePanel.tsx     state A (upload / paste)
├── AIReviewProgressPanel.tsx   state B (step progress)
├── AIReviewPanel.tsx           state C shell
│   ├── TranscriptPanel.tsx     collapsible segment list
│   ├── SoapDraftForm.tsx       4 textareas + Accept/Reject buttons
│   └── CodeSuggestionList.tsx  code checkboxes grouped by type
├── shared.tsx                  ConfidencePill, WarningsBanner, AIReviewSummaryCard
├── actions.ts                  "use server" — 5 actions
└── getAIReviewContext.ts       server-side data fetcher
```

---

3. State transitions

Session state (DB, already modeled on `ai_scribe_sessions.status`):

```
pending → transcribing → generating → suggesting_codes → awaiting_review
                                                          │
                                                ┌─────────┼─────────┐
                                                ▼         ▼         ▼
                                            accepted  rejected  cancelled
any non-terminal → failed
```

UI state selection (derived):

Session state	UI state	Actions offered
none	A (Intake)	`startSession`
pending, transcribing, generating, suggesting_codes	B (Progress)	`refresh`, `cancelSession`
awaiting_review	C (Review)	`acceptDraft`, `rejectDraft`, `cancelSession`
failed	B-error	`retryStep`, `cancelSession`
accepted	D-success	`viewDetails`
rejected, cancelled	D-terminal	`startNewSession`

---

4. Data model interactions

Reads — single `getAIReviewContext(encounterId, ctx)` helper

Four queries in one render pass:

1. `ai_scribe_sessions` — most recent for `(encounter_id, tenant_id)`.
2. `ai_scribe_transcript_segments` — for the session, ordered by `sequence_index`.
3. `ai_completions.structured_output` — for the session's generate-step `request_id` (the SOAP draft).
4. `ai_scribe_code_suggestions` — for the session, ordered by `type, rank`.

Writes — server actions

Action	Writes
`startSession`	INSERT `ai_scribe_sessions` (status=pending); returns signed `uploadUrl` for audio when `source=audio_upload`. Emits `ai.scribe_session_created`.
`submitTranscript`	Paste: run `TranscriptionService.chunkText` inline + INSERT segments. Audio: mark step `transcribing`, orchestrator owns the rest. Emits `ai.transcript_submitted`. Kicks the SOAP + codes pipeline.
`refresh`	No writes; server component re-reads on render.
`acceptDraft`	INSERT `encounter_notes` (status= `draft`, `ai_assisted=true`, `ai_request_id` set). UPDATE `ai_scribe_code_suggestions.accepted_at` + `accepted_by` for the accepted ids. INSERT `diagnosis_assignments` for accepted ICD-10 codes. UPDATE session → `accepted`. Emits `ai.draft_accepted`.
`rejectDraft`	INSERT `ai_feedback` row with reason + optional correction. UPDATE session → `rejected`. Emits `ai.draft_rejected`.
`cancelSession`	UPDATE session → `cancelled`.

All writes use the admin Supabase client inside server actions, guarded by explicit tenant + permission checks — matches the existing `saveNoteDraft` / `signNote` / `amendNote` pattern.

---

5. Validation rules

Client-facing guards + server re-validation (server action is always source of truth):

Rule	Where
At least one SOAP section non-placeholder before accept	server action; `?error=empty_draft`
≤ 30 accepted codes per accept action	server action
Reject reason min 5 chars	`minlength=5` on input + server re-check
No accept if current note on encounter is already signed	server action returns 409-equivalent; banner surfaced

No transcript submit if session not in `pending`	server action; `?error=bad_state`
Audio upload ≤ 50 MB, duration ≤ 60 min, mime `audio/*`	signed-URL policy + server re-check
Pasted transcript between 10 and 200,000 chars	`SubmitTranscriptInputSchema` (Zod)
`acceptedCodes[]` must all belong to this session	server action FK + session_id check
User can't accept while impersonating	server action refuses; banner

---

6. Audit events

Identical taxonomy to docs/ai-scribe.md §9. Emission points from the UI layer:

Event	Emitter
`ai.scribe_session_created`	`startSession` action
`ai.transcript_submitted`	`submitTranscript` action
`ai.draft_generated`	orchestrator (SoapDraftService caller), not UI
`ai.codes_suggested`	orchestrator, not UI
`ai.draft_accepted`	`acceptDraft` action
`ai.draft_rejected`	`rejectDraft` action
`ai.invocation_blocked`	orchestrator / guardrails
`ai.hallucinated_trace`	service layer (already emitted inside `sanitizeSegmentIds` callers)

All via `logEventBestEffort` from `@vitalflow/auth/audit`.

---

7. Permissions

Capability	Required
See the AI Review card	`ai:invoke`
Start a new session	`ai:invoke` + encounter in user's tenant
Cancel active session	`ai:invoke` + (session creator OR `admin:tenant`)
Accept draft	`ai:invoke` + `clinical:write` + not impersonating
Reject draft	`ai:invoke`
Sign the accepted note	`clinical:sign` (on the existing Clinical-note card — unchanged)

Permissions are resolved once in the server component and passed to children as booleans (`canAcceptDraft` , `canCancel` , ...). No child re-checks.

---

8. Acceptance criteria

- [ ] User with `ai:invoke` on an active encounter sees the AI review card; user without it does not see it at all.
- [ ] Starting a paste session with ≥10 chars creates a session and triggers the pipeline; progress state renders within 1s.
- [ ] During states `pending|transcribing|generating|suggesting_codes` , the card shows step status. Refresh does not lose textarea edits because edits are form-local and only posted on Accept/Reject.
- [ ] When `awaiting_review` , the card renders all 4 SOAP sections with editable textareas prefilled from the draft, a confidence pill colored by `combined` , and a source list rendered as short segment-id prefixes.
- [ ] Clicking a source pill scrolls the transcript panel to the cited segment (pure `<a href="#seg-...">` , no JS).
- [ ] Code suggestions with `combined < 0.5` are hidden by default; a "Show low-confidence suggestions" toggle (URL query string) reveals them.
- [ ] Accept flow: creates `encounter_notes` row with `ai_assisted=true` , `ai_request_id` set, status= `draft` ; session → `accepted` ; selected codes get `accepted_at` ; ICD-10 codes materialize as `diagnosis_assignments` .
- [ ] Reject flow: writes `ai_feedback` row; session → `rejected` ; card transitions to terminal summary.
- [ ] Version history on the existing Clinical-note card shows the new note with an "AI" pill.
- [ ] Impersonating user cannot accept; server action refuses; user-facing banner.
- [ ] Encounter with a signed current note refuses accept with a 409-equivalent banner.
- [ ] Rejecting twice in a row does not crash; second returns "session already rejected".
- [ ] All actions emit their audit event.
- [ ] Server-side typecheck, vitest, and Next.js build all pass without `"use client"` being introduced.

---

9. Not V1

- **Per-section accept/reject** — V1 is per-draft. Sections have inline edit; one Accept writes all four.
- **Real-time progress streaming** — V1 uses a `Refresh` button in state B. V2 can add SSE or a client component that polls.
- **Inline transcript audio playback** — V1 shows text segments; audio-segment sync is later.

- **Physician-side rationale editing on code suggestions** — V1 shows rationale/missing-docs read-only.
- **Multi-session history per encounter** — V1 shows only the most recent session. An "older sessions" drawer comes later.